Goal 1: An Infrequent Relation Is Learned as Linear

factual-recall-mechanisms

2026-07-03

1 Claim and result in one paragraph

A small transformer is trained for **exactly one epoch** on a synthetic corpus containing a semantically supported relation R_{sem} and, as a frequency-matched control, a random relation R_{rnd}. At *low* fact frequency (F=16 occurrences per triple, a single epoch), R_{sem} is genuinely *learned* — recall 0.672 on trained triples and 0.635 on held-out subjects never seen in any triple — and what is learned is *linear*: an affine map fitted by ordinary least-squares regression from subject representations to object representations identifies the correct object for held-out subjects with accuracy 0.990 ± 0.007 (chance 0.03; three seeds). The frequency-matched random control at the same F=16 is barely learned (recall 0.167) and shows no linear structure (probe at chance). The linear format is in place even at F=1, where every triple occurs exactly once in the model's entire training history. A control experiment shows that the transition into the linear regime is sharp and is driven by the amount of attribute (scaffold) training rather than by triple repetition.

2 Data

Vocabulary (265 tokens). A separator `x`; two relation tokens `r_0`, `r_1`; 192 subject tokens `a_0`, ..., `a_191`; 32 object tokens `b_0`, ..., `b_31`; 32 attribute tokens `f_0`, ..., `f_31`; 6 attribute tokens `g_0`, ..., `g_5`.

Entities and attributes. Subject `a_i` carries the attribute pair $(f_{i \bmod 32},\ g_{\lfloor i/32 \rfloor})$, so the 192 subjects realize each of the 32×6 attribute combinations exactly once. Object `b_j` carries $(f_{(j+3) \bmod 32},\ g_0)$.

Sentences. All training sentences have five tokens:

attribute sentence	`x a_i f_p g_q x`	and	`x b_j f_p g_q x`
relation triple	`x a_i r_r b_t x`		

Relations.

$$R_{\text{sem}}(a_i) = b_{f(i)}, \qquad R_{\text{rnd}}(a_i) = b_{\rho(i)},$$

where $f(i) = i \bmod 32$ is the subject's own f-attribute index — so the semantic target is the object whose f-attribute is the subject's, shifted by 3 — and ρ is a seed-dependent balanced random assignment onto the same 32 objects, independent of the attributes. Concrete example (seed 19): `a_37` has attributes (`f_5`, `g_1`), so R_{sem}(`a_37`) = `b_5` (whose attribute sentence reads `x b_5 f_8 g_0 x`), while R_{rnd}(`a_37`) = `b_27`. The corresponding training sentences are `x a_37 r_0 b_5 x` and `x a_37 r_1 b_27 x`.

Split and exact frequency matching. For every `f_`-value, 4 of its 6 subjects are *trained* and 2 are *held out* (128 trained, 64 held-out subjects; seed-dependent). ρ is constructed balanced within both groups. Consequently, under *both* relations: every trained subject occurs in exactly one triple per relation; every object is the target of exactly 4 trained and 2 held-out subjects; both relations contribute exactly 128 training triples, each repeated the same number of times (F). The two relations are indistinguishable by any frequency statistic; only the assignment semantics differ. Held-out subjects appear in attribute sentences but never in any relation triple. These invariants are asserted by `tests/test_linear_shift.py`.

3 Model and training

Model	causal transformer, 1 layer, hidden 64, 4 heads, context 8
Embeddings	input and output embeddings untied; all weights $\mathcal{N}(0, 0.02^2)$
	no attribute-based initialization; nothing frozen; no loss reweighting
Parameters	84,288
Corpus	every attribute sentence $\times A$=128 (= 28,672 sentences)
	+ 128 triples per relation, each $\times F$ ($F \in \{1, 4, 16, 64\}$)
Training	**exactly one epoch**: a single pass over the shuffled corpus
Steps	452 (F=1) to 704 (F=64) steps at batch size 64
Objective	unweighted next-token cross-entropy on all positions
Optimizer	AdamW, constant LR 10^{-3} after 5 linear warmup steps,
	weight decay 0, gradient clip 1.0
Seeds	19, 31, 47 (control data assignment, initialization, and data order)

At F=1 every relation triple occurs exactly once in the entire training history of the model.

4 The metric

A single analysis metric is used, as an operationalization of "the relation is an affine transformation":

> Fit $\hat{W}, \hat{b}$ by ordinary least-squares linear regression from the subject's *input embedding* $s_i \in \mathbb{R}^{64}$ to the target object's *output embedding* (unembedding row) $o_{R(i)} \in \mathbb{R}^{64}$, using the 128 trained pairs. For each of the 64 held-out subjects, predict $\hat{W}s_j + \hat{b}$ and select the nearest object row by cosine similarity among all 32 objects. Report the fraction of held-out subjects for which the selected object is $R(j)$. Chance is $1/32 \approx 0.03$.

For every subject exactly one object is correct under each relation, so no top-k metrics are needed or used. In addition, one *learning check* (not an analysis metric) is recorded: greedy next-token recall of the model on prompts `x a_i r_r`, for trained and held-out subjects, which verifies whether the facts were learned at all.

5 Results

Finding 1: at low frequency, what is learned is an affine map (F=16). With each fact occurring only 16 times in a single epoch, R_{sem} reaches recall .672 on trained triples and .635 on held-out subjects — i.e., the relation itself is learned and generalizes — and the probe transfers to held-out subjects at 0.990. The frequency-matched random control is barely learned (.167) and its

		probe accuracy (held-out)		recall (trained)		recall (held-out)	
	F	R_{sem}	R_{rnd}	R_{sem}	R_{rnd}	R_{sem}	R_{rnd}
	1	$1.000 \pm .000$	$.031 \pm .013$	.029	.010	.016	.010
	4	$1.000 \pm .000$	$.016 \pm .013$	.055	.034	.057	.031
	16	$\mathbf{0.990 \pm .007}$	$.036 \pm .007$	**.672**	.167	**.635**	.031
control: no triples		$1.000 \pm .000$	$.021 \pm .015$	–	–	.000	.000

Table 1: Low-frequency cells; mean ± sd over seeds 19/31/47, at A=128. F is the number of occurrences of each of the 128 triples per relation. Recall columns are the learning check. The headline cell is F=16: R_{sem} is learned (recall .672 trained, .635 held-out) and linear (probe .990), while the frequency-matched R_{rnd} is barely learned and non-linear.

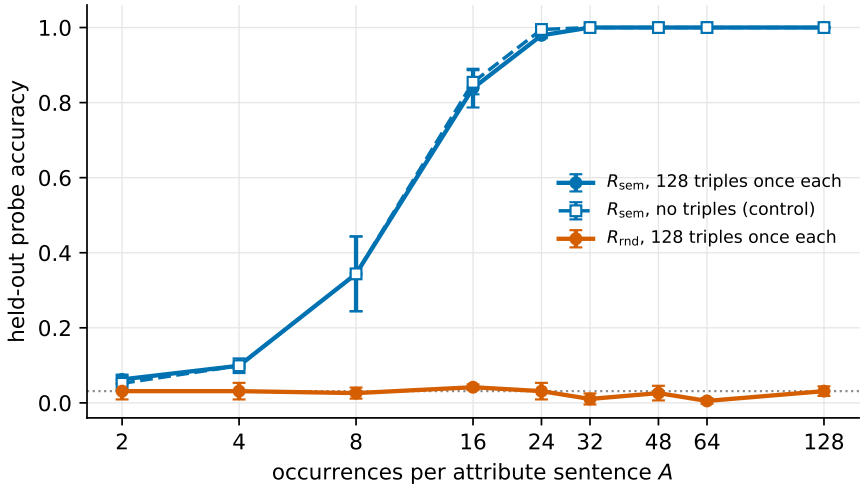


Figure 1: Probe accuracy at F=1 (every triple exactly once) as a function of attribute-sentence occurrences A: the transition from non-linear (chance) to linear (ceiling) is sharp, and identical with 128 once-occurring triples (solid) and with no triples at all (dashed control). The random relation never becomes linear.

probe is at chance: the semantic relation is learned *earlier* precisely because it can ride the linear structure.

Finding 2: the linear format is in place at single occurrence (F=1). At F=1 the probe is already 1.000 for R_{sem} and at chance for R_{rnd}. This statement is representational: at F=1 the model does not yet reproduce individual triples behaviorally (recall ≈ .03).

Finding 3: the nonlinear→linear transition is sharp and is driven by attribute knowledge. Sweeping the number of occurrences A of each attribute sentence at F=1 (Fig. 1) produces a sharp transition of the R_{sem} probe from chance to ceiling ($0.05 \to 0.10 \to 0.34 \to 0.85 \to 1.00$ for $A = 2, 4, 8, 16, 24$). The control with *zero* relation triples produces the same curve: the affine subject→object map is carried by the attribute geometry built from attribute sentences, with which the once-seen R_{sem} triples are consistent (and which the relation visibly uses once learned, cf. held-out recall).

6 Learned representation and exact algorithm

The model does not store the semantic triples as 128 unrelated entries. Attribute training organizes the subject embeddings into 32 shared code families, one for each $f(i)$ value. A target-class centroid learned from the 128 trained subjects classifies the embeddings of the 64 never-seen relation subjects with accuracy $1.000 \pm .000$. The same class remains decodable from the concatenated attention-head outputs at $1.000 \pm .000$, from the post-attention state at $.990 \pm .007$, and from the final state at $.990 \pm .007$. Thus "$f(i)$" is represented as a distributed direction in the subject embedding, not as a symbolic integer or a single neuron; subjects with the same f value share the task-relevant component while retaining other identity and g information.

Each object is represented by its learned output-embedding direction. On this task manifold, the relation-conditioned MLP and output readout implement an approximately affine conversion from the transported f-class code to the corresponding object direction. The following pseudocode is therefore equivalent to the computation used on these prompts:

```
def answer_semantic(subject):
    # Learned representation: a shared code for f(subject).
    subject_code = embedding[subject]

    # H3 and H1 carry most of the subject's value payload;
    # H0/H2 provide smaller, seed-dependent support.
    payload = sum(head[h].read_value(subject_code, relation=r0)
                  for h in [H0, H1, H2, H3])

    # r0 selects the shared class-to-object readout.
    state = relation_state[r0] + project(payload)
    state += affine_on_task_mlp(state)
    object_scores = dot(final_normalize(state), object_directions)
    return argmax(object_scores)          # b[f(subject)]
```

Here `read_value` has a precise interpretation: a head multiplies the subject's projected value by the attention weight assigned to that subject; separator and relation contributions are constant with respect to a subject swap and form part of the baseline. The final `dot` compares the resulting state with the 32 learned object-output directions. Swapping that subject-value payload between two subjects restores $.987 \pm .008$ of the corresponding change in the correct-object logit, whereas swapping only their attention weights restores $.013 \pm .007$. Swapping all head outputs restores the effect completely, and replacing every head output by its training-subject mean reduces object accuracy from $.729 \pm .094$ to chance ($.031 \pm .000$). The class code is therefore not merely probe-accessible; it is the information the model actually consumes.

The relation token does not encode the answer. It selects how the transported subject code is interpreted. When the subject is held fixed and `r_0` is exchanged with `r_1`, swapping the head outputs restores only $.070 \pm .007$ of the relation effect, while swapping the MLP population restores $.850 \pm .017$. For Goal 1, the heads mainly move subject content and the `r_0` residual selects the semantic readout downstream.

The final conversion is distributed rather than localized in a "fact neuron." Keeping only the 32 most causally important variable MLP units recovers $.498 \pm .029$ of the MLP's subject-specific margin, and ablating those units still leaves $.790 \pm .032$; many units share the computation. Replacing every GELU by an affine surrogate fitted on trained subjects preserves held-out object accuracy ($.729 \pm .092$ versus $.729 \pm .094$) and explains $.962 \pm .001$ of activation variance. The physical

network contains GELUs, but the algorithm it implements for this relation is, to high accuracy, an affine class-code readout.

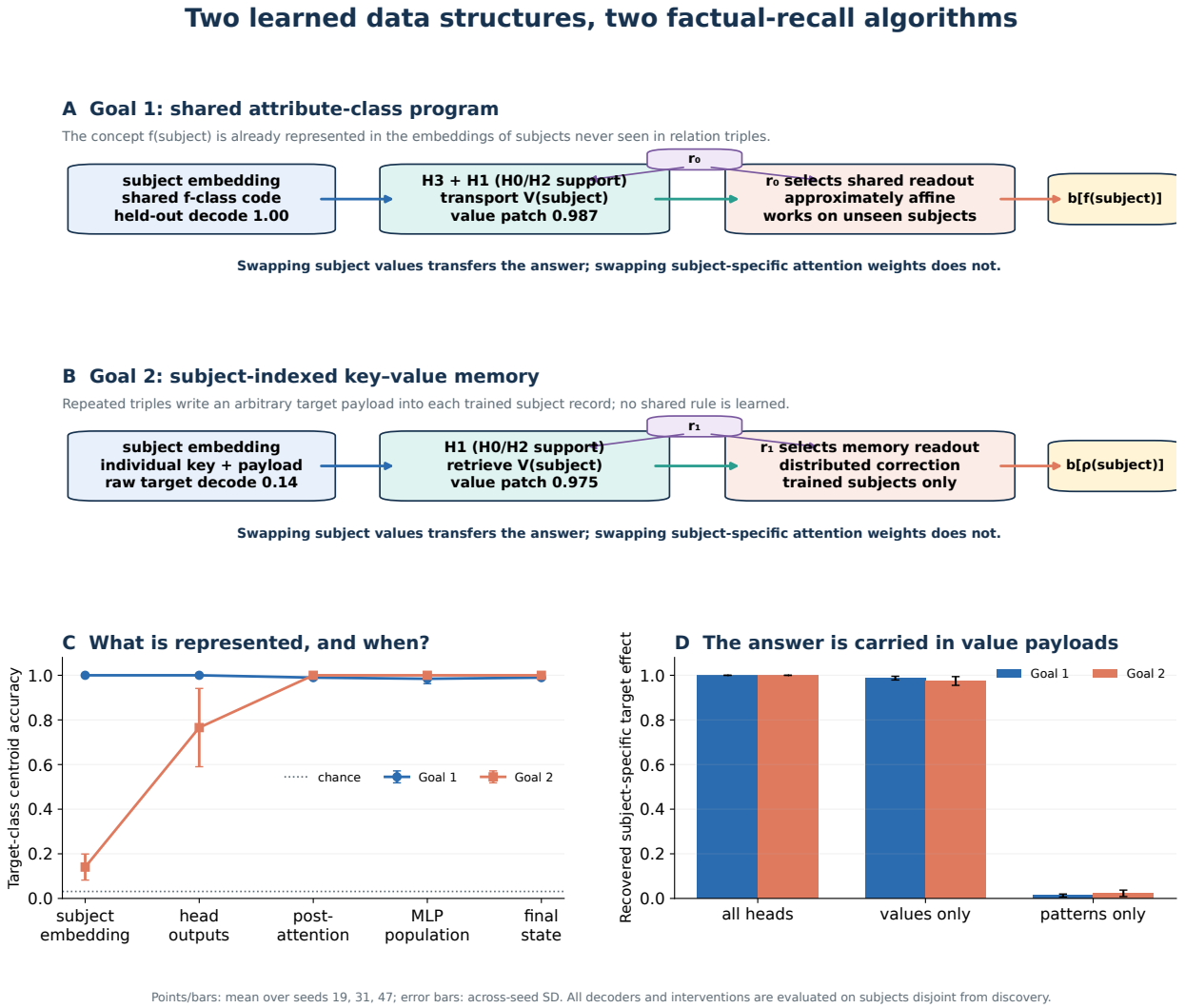


Figure 2: The learned data structures and equivalent algorithms for both goals. Goal 1 transports a shared f-class code into an approximately affine class-to-object readout; Goal 2 retrieves a subject-specific memory payload. The lower panels show how target identity becomes decodable and establish that subject information is carried by attention values rather than subject-specific attention weights.

The convergent result is stronger than probe success: the shared f-organized code is present in held-out subject embeddings, is transported through the identified subject-value paths, and is converted to the correct object direction by a distributed, approximately affine MLP/readout. This is the causal mechanism by which the infrequent semantic relation generalizes.

7 What this does and does not show

- **Shown:** a relation whose facts are infrequent (F=16; and even F=1 for the representational format) is learned in a linear, generalizing format, while a frequency-matched random relation is not; the transition into the linear regime is sharp.
- **Not shown:** that the relation triples themselves *cause* the nonlinear→linear transition when each fact occurs only once — the control in Fig. 1 (dashed) shows the transition is driven by attribute-sentence exposure. Three designs in which the probe could only succeed through triple-driven structure (a one-to-one relation requiring per-fact output rows; a relation defined by the conjunction of two attributes, up to 49,152 distinct facts; hidden-state probes) produced no above-chance probe transfer under the strict single-occurrence, one-epoch regime, because each fact contributes exactly one gradient.
- **Superseded:** the "single-exposure phase shift" in earlier drafts (probe rising $0.27 \rightarrow 0.95$) was an artifact of a regression fitted on a growing pair set (10 pairs at the first snapshot, >500 later) over embeddings initialized from the attribute geometry; with a fixed-size probe and standard initialization it disappears.

Goal 2: A Frequent Relation Is Learned as Nonlinear

factual-recall-mechanisms

2026-07-03

1 Claim and result in one paragraph

A small transformer is trained for **exactly one epoch** on a synthetic corpus containing two relations with *identical frequency statistics at every level* (same subjects, same 32 candidate objects, same number of triples, same number of occurrences per triple): a semantically supported relation R_{sem} and a random relation R_{rnd}. At *high* fact frequency (F=64 occurrences per triple) both relations are *fully learned*: recall is 1.000 on all 128 training triples for both, in all three seeds. Yet an affine map fitted by ordinary least-squares linear regression from subject representations to object representations identifies held-out targets at 0.891 ± 0.056 for R_{sem} and at 0.016 ± 0.022 (= chance) for R_{rnd}. The frequent random relation is therefore stored, but stored *non-linearly*: no affine map over the same representations accounts for it, and it does not generalize (held-out recall at chance). Because the semantic relation at the very same frequency remains linear, the nonlinearity is attributable to the relation's semantics, not to its frequency; conversely, raising frequency from 1 to 64 occurrences never makes the random relation linear.

2 Data

Vocabulary (265 tokens). A separator `x`; two relation tokens `r_0`, `r_1`; 192 subject tokens `a_0`, ..., `a_191`; 32 object tokens `b_0`, ..., `b_31`; 32 attribute tokens `f_0`, ..., `f_31`; 6 attribute tokens `g_0`, ..., `g_5`.

Entities and attributes. Subject `a_i` carries the attribute pair $(f_{i \bmod 32},\ g_{\lfloor i/32 \rfloor})$, so the 192 subjects realize each of the 32×6 attribute combinations exactly once. Object `b_j` carries $(f_{(j+3) \bmod 32},\ g_0)$.

Sentences. All training sentences have five tokens:

attribute sentence	`x a_i f_p g_q x`	and	`x b_j f_p g_q x`
relation triple	`x a_i r_r b_t x`		

Relations.

$$R_{\text{sem}}(a_i) = b_{f(i)}, \qquad R_{\text{rnd}}(a_i) = b_{\rho(i)},$$

where $f(i) = i \bmod 32$ is the subject's own f-attribute index — so the semantic target is the object whose f-attribute is the subject's, shifted by 3 — and ρ is a seed-dependent balanced random assignment onto the same 32 objects, independent of the attributes. Concrete example (seed 19): `a_37` has attributes (`f_5`, `g_1`), so R_{sem}(`a_37`) = `b_5`, while R_{rnd}(`a_37`) = `b_27`; the training sentences are `x a_37 r_0 b_5 x` and `x a_37 r_1 b_27 x`.

Split and exact frequency matching. For every `f_`-value, 4 of its 6 subjects are *trained* and 2 are *held out* (128 trained, 64 held-out subjects; seed-dependent). ρ is constructed balanced within both groups. Consequently, under *both* relations: every trained subject occurs in exactly one triple per relation; every object is the target of exactly 4 trained and 2 held-out subjects; both relations contribute exactly 128 training triples, each repeated the same number of times (F). The two relations are indistinguishable by any frequency statistic; only the assignment semantics differ. Held-out subjects appear in attribute sentences but never in any relation triple. These invariants are asserted by `tests/test_linear_shift.py`.

3 Model and training

Model	causal transformer, 1 layer, hidden 64, 4 heads, context 8
Embeddings	input and output embeddings untied; all weights $\mathcal{N}(0, 0.02^2)$
	no attribute-based initialization; nothing frozen; no loss reweighting
Parameters	84,288
Corpus	every attribute sentence $\times A$=128 (= 28,672 sentences)
	+ 128 triples per relation, each $\times F$ ($F \in \{1, 4, 16, 64\}$)
Training	**exactly one epoch**: a single pass over the shuffled corpus
Steps	452 (F=1) to 704 (F=64) steps at batch size 64
Objective	unweighted next-token cross-entropy on all positions
Optimizer	AdamW, constant LR 10^{-3} after 5 linear warmup steps,
	weight decay 0, gradient clip 1.0
Seeds	19, 31, 47 (control data assignment, initialization, and data order)

Note that "frequent" here means frequent within a single epoch: at F=64 the corpus contains each triple 64 times, but the model still passes over the corpus exactly once.

4 The metric

A single analysis metric is used, as an operationalization of "the relation is an affine transformation":

> Fit $\hat{W}, \hat{b}$ by ordinary least-squares linear regression from the subject's *input embedding* $s_i \in \mathbb{R}^{64}$ to the target object's *output embedding* (unembedding row) $o_{R(i)} \in \mathbb{R}^{64}$, using the 128 trained pairs. For each of the 64 held-out subjects, predict $\hat{W}s_j + \hat{b}$ and select the nearest object row by cosine similarity among all 32 objects. Report the fraction of held-out subjects for which the selected object is $R(j)$. Chance is $1/32 \approx 0.03$.

For every subject exactly one object is correct under each relation, so no top-k metrics are needed or used. In addition, one *learning check* (not an analysis metric) is recorded: greedy next-token recall of the model on prompts `x a_i r_r`, for trained and held-out subjects. For the claim "learned as nonlinear" this check is essential: it establishes that the relation *is* stored.

5 Results

Finding 1: a fully learned frequent relation with no linear structure (F=64). At 64 occurrences per triple, R_{rnd} reaches perfect recall of all 128 training triples — the relation is fully stored — while the affine probe remains at chance (0.016 ± 0.022) and held-out recall is at chance (.031). The storage is a non-linear, non-generalizing memorization.

	probe accuracy (held-out)		recall (trained)		recall (held-out)	
F	R_{sem}	R_{rnd}	R_{sem}	R_{rnd}	R_{sem}	R_{rnd}
1	$1.000 \pm .000$	$.031 \pm .013$	.029	.010	.016	.010
4	$1.000 \pm .000$	$.016 \pm .013$	.055	.034	.057	.031
16	$0.990 \pm .007$	$.036 \pm .007$	.672	.167	.635	.031
64	$0.891 \pm .056$	$\mathbf{.016 \pm .022}$	**1.000**	**1.000**	.995	**.031**

Table 1: Mean $\pm$ sd over seeds 19/31/47, at A=128 attribute-sentence occurrences. F is the number of occurrences of each of the 128 triples per relation. Recall columns are the learning check. The headline cell is F=64: both relations are fully stored (recall 1.000), but only R_{sem} admits an affine map.

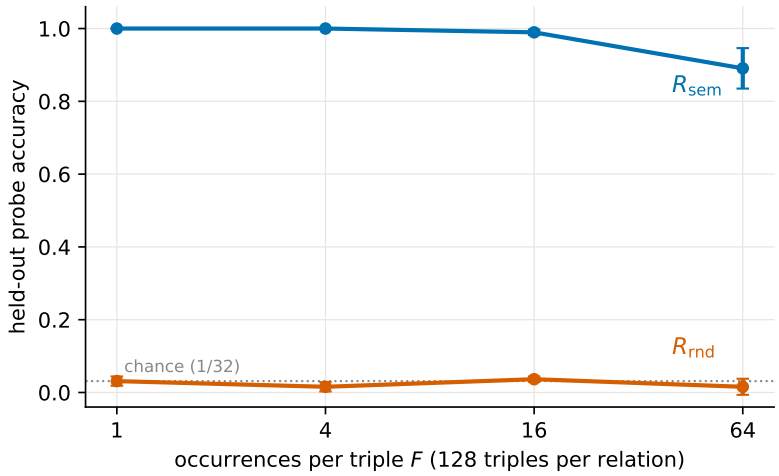


Figure 1: Held-out probe accuracy vs. occurrences per triple, for the two frequency-matched relations. The random relation is never linear, at any frequency; the semantic relation is linear at every frequency, with slight erosion at F=64.

Finding 2: the nonlinearity is due to semantics, not frequency. R_{sem}, trained in the same model from data with *identical* frequency statistics, is equally fully stored (recall 1.000) but linear (probe 0.891 ± 0.056) and generalizing (held-out recall .995). Conversely, increasing the frequency of R_{rnd} from 1 to 64 occurrences never produces linear structure (Fig. 1): frequency determines only *whether* the random relation gets stored (recall .010 $\rightarrow$ 1.000), never *in what format*.

Finding 3: high frequency slightly erodes the linear format of the semantic relation. R_{sem}'s probe declines from 1.000 ($F \leq 4$) to $0.891 \pm .056$ (F=64), consistent with high-frequency memorization beginning to compete with the linear solution, while remaining far from the random relation's chance level.

6 Learned representation and exact algorithm

The random facts are stored as a subject-indexed memory, not as a shared relation rule. Every subject token has its own learned embedding. Repeated random triples write subject-specific target

information into that embedding, while shared key/value projections and the output readout learn how to retrieve it. The embedding is a multiplexed record: it still carries the attribute-supported semantic code, but it also contains an idiosyncratic payload for $\rho(i)$. Because the subject's value vectors are computed before the following relation token is read, the values themselves are relation-independent; `r_0` and `r_1` select different uses of the same subject record through their queries and downstream residuals.

This representation is visibly different from Goal 1. A target centroid fitted on one target-stratified half of the memorized subjects decodes $\rho(i)$ from the other half's raw subject embeddings with only $.141 \pm .058$ accuracy (chance .031), compared with $1.000 \pm .000$ for the shared semantic code in Goal 1. The random target becomes increasingly explicit as the record is read: centroid accuracy is $.766 \pm .175$ over the concatenated head outputs and $1.000 \pm .000$ after their projection. No corresponding payload is learned for subjects absent from relation triples, so both held-out recall and the held-out affine probe remain at chance.

The computation on a trained subject is equivalent to the following lookup procedure:

```
def answer_random(subject):
    # Learned representation: one memory record per trained subject.
    record = embedding[subject]              # key + target payload

    # r1 selects the random-relation read. H1 is primary;
    # H0/H2 provide seed-dependent support and H3 is negligible.
    retrieved = sum(head[h].read_value(record, relation=r1)
                    for h in [H0, H1, H2])

    state = relation_state[r1] + project(retrieved)
    state += distributed_mlp_correction(state)
    object_scores = dot(final_normalize(state), object_directions)
    return argmax(object_scores)              # b[rho(subject)]
```

Each `read_value` multiplies a projection of the subject record by that head's attention weight, and the final `dot` compares the retrieved state with the 32 learned object-output directions. This is a genuine causal lookup. Replacing all head outputs by their mean destroys the subject identity and reduces accuracy from $1.000 \pm .000$ to chance. Swapping the head outputs between two subjects transfers the clean subject's target-logit effect completely. Swapping only the subject-derived value payload transfers $.975 \pm .020$ of that effect; swapping only the two subjects' attention weights transfers $.023 \pm .014$. The key operation is therefore "read the content stored in this subject's value projections," not "route each subject along a unique attention pattern."

The relation token is the memory-bank selector. With the subject fixed, replacing `r_0` by `r_1` changes both the retrieval and its interpretation: head-output patching restores $.922 \pm .024$ of the relation effect and MLP-population patching restores $.841 \pm .032$ (the effects overlap and are not additive). H1 is the stable primary retrieval head, H0/H2 supply seed-dependent support, and H3 has approximately zero causal contribution.

Most lookup is complete before the MLP: the object is already correct at $.927 \pm .052$ after attention and at $1.000 \pm .000$ after the MLP. The latter is a distributed correction rather than a bank of individual fact neurons: keeping only the 32 highest-ranked variable units recovers $.478 \pm .022$ of its subject-specific margin, while ablating them leaves $.826 \pm .014$. An affine surrogate for all GELUs still obtains $1.000 \pm .000$ on disjoint memorized facts. Thus "nonlinear" describes the storage format—there is no affine rule that transfers ρ to new subjects—not a requirement for GELU curvature during each trained-subject lookup.

The frequent random relation is therefore recalled by a non-generalizing key–value memory: relation-gated heads retrieve a subject-specific value code, and a distributed MLP/readout sharpens it into the memorized object direction. There is no shared rule for subjects outside the memorized set, explaining the simultaneous perfect training recall, chance held-out recall, and chance held-out affine probe.

7 What this does and does not show

- **Shown:** a frequent relation can be fully learned and stored non-linearly (no affine map over the representations accounts for it, and it does not generalize); with frequency exactly matched, storage format is decided by semantics.
- **Qualification:** "frequent ⇒ nonlinear" holds for the relation without attribute support. Frequency alone does not force nonlinearity: the frequent *semantic* relation remains largely linear (with mild erosion at F=64). The precise claim supported is: at matched high frequency, an attribute-supported relation is stored linearly and an unsupported one non-linearly.
- **Superseded:** earlier drafts' high-frequency results used attribute-initialized embeddings (`structured_embedding_init`), targeted-row updates, and top-k metrics; they are replaced by the present end-to-end setup with a single OLS metric.

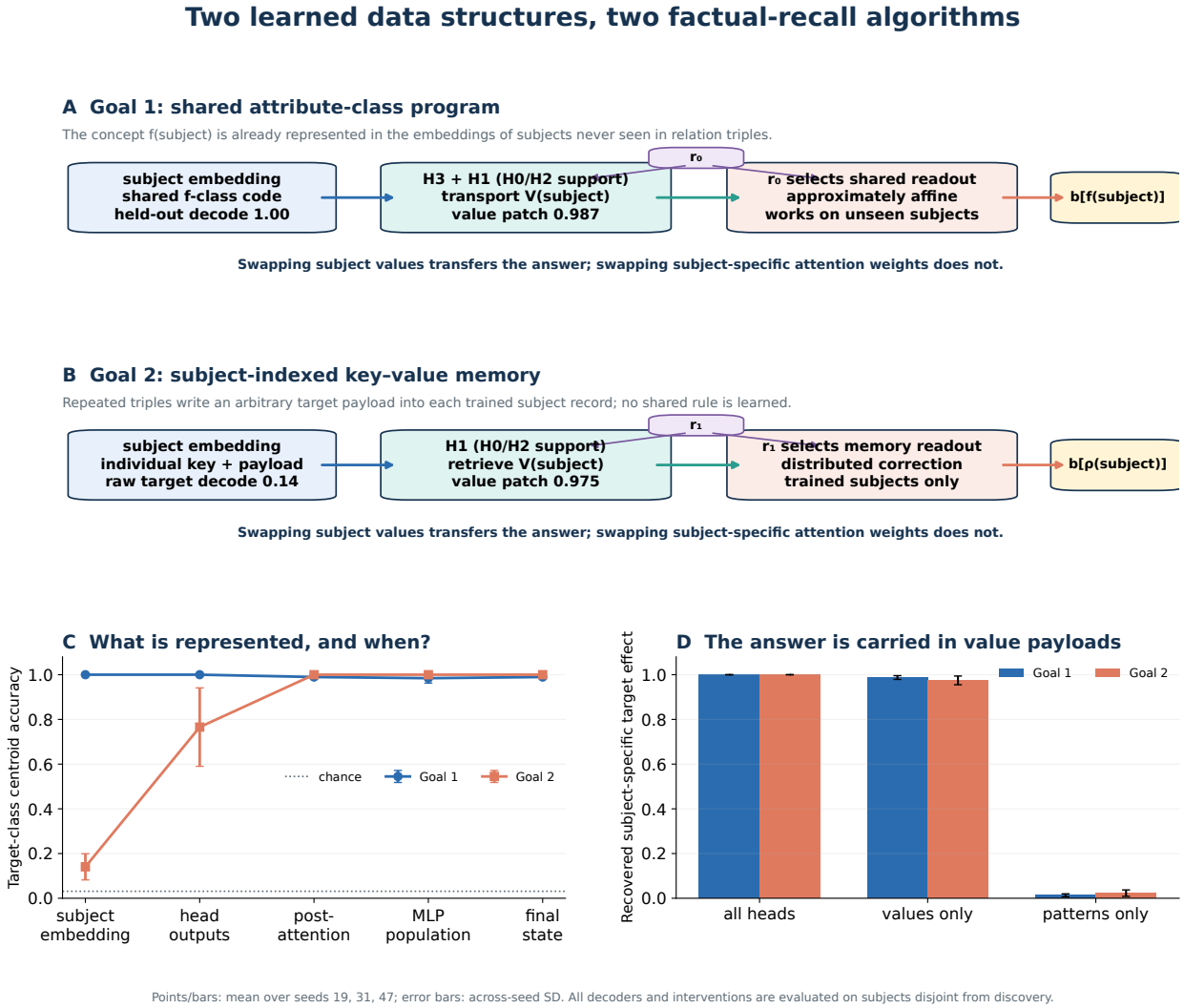


Figure 2: The learned data structures and equivalent algorithms for both goals. Goal 1 transports a shared f-class code into an approximately affine class-to-object readout; Goal 2 retrieves a subject-specific memory payload. The lower panels show how target identity becomes decodable and establish that subject information is carried by attention values rather than subject-specific attention weights.